NATIONAL PARCEL AND FREIGHT CARRIER OPERATIONAL DATABASE

Database Schema Documentation

100/100 Quality Score	Grade A Schema Grade	35 Tables Generated	28 Rules Applied

Generated On	September 05, 2026 at 10:19
Domain	Logistics
Scale	Enterprise
GST Compliance	Yes
AI Provider	Together — openai/gpt-oss-120b

1. PROJECT OVERVIEW

Provide a high■performance, end■to■end system for tracking, routing, billing, and exception handling of millions of shipments per month.

Modules

1. Infrastructure	System registry, unique ID mapping, and configuration
`unique_id_header_all`	Centralised business ID registry for tracking every entity across all modules
`factory_reset_all`	Global platform feature flags and kill-switches singleton

2. Shipment Management	Core domain handling creation, routing, tracking, and lifecycle of shipments.
`shipment_request_header_all`	Input data collected when a shipper creates a new shipment request (origin, destination, weight, dimensions, service level).
`shipment_request_archive_all`	Historical archive mirror of shipment_request_header_all
`shipment_header_all`	Master record representing a parcel or freight shipment moving through the network.
`shipment_archive_all`	Historical archive mirror of shipment_header_all
`shipment_life_cycle_all`	State transition log and audit trail for shipment_header_all
`route_plan_details_all`	Calculated routing information including optimal path, carrier assignment, and any alternative plans.
`shipment_event_transaction_all`	Individual tracking event (scan, GPS ping, status change) generated by carriers or hubs.
`shipment_tracking_details_all`	Aggregated history of ShipmentEvents for a given Shipment, used for current location and ETA calculations.
`shipment_status_lookup_all`	Lookup table of possible shipment status values (InTransit, Delivered, Delayed, etc.).
`exception_record_transaction_all`	Record of an exception detected during shipment tracking (delay, damage, customs hold, etc.) with classification and severity.
`exception_record_life_cycle_all`	State transition log and audit trail for exception_record_transaction_all
`exception_type_lookup_all`	Lookup table defining exception categories and severity levels.
`route_plan_exception_all`	Link entity capturing alternative route plans generated when an exception occurs.

3. Finance & Billing	Generation of invoices, ledger postings, and payment processing for shipments.
`invoice_transaction_all`	Financial document generated for a Shipment containing line items, taxes, duties and total amount due.
`invoice_life_cycle_all`	State transition log and audit trail for invoice_transaction_all
`ledger_entry_transaction_all`	Debit or credit posting representing the financial impact of a shipment charge, payment, or adjustment.

`audit_log_transaction_all`	Immutable log entry capturing who changed what, when, and how across the system for compliance and audit purposes.
4. Exception & Approval	Detection, classification, and resolution of shipment exceptions with approval workflow.
`approval_task_details_all`	Workflow task representing a single approval step in the exception resolution process.
5. User & Security	Management of system users, roles, and API credentials.
`user_account_header_all`	Master entity for system users, storing identity, role assignments, and organization membership.
`user_account_archive_all`	Historical archive mirror of user_account_header_all
`api_credential_details_all`	Authentication credential (API key, token) issued to a UserAccount for programmatic access.
6. Notification Service	Generation and delivery of notifications to customers and internal users.
`notification_event_transaction_all`	Trigger source indicating that a notification should be sent (e.g., status change, invoice generated).
`notification_message_details_all`	Formatted message ready for delivery, including template, channel, recipient, and payload.
`notification_channel_lookup_all`	Lookup of supported notification channels (Email, SMS, Push).
7. Reporting & Analytics	On■demand and scheduled reporting, analytics, and data export.
`report_request_header_all`	User■initiated request specifying report type, filters, and output format.
`report_request_archive_all`	Historical archive mirror of report_request_header_all
`report_result_details_all`	Generated dataset or visualization produced by the analytics engine for a ReportRequest.
`scheduled_report_header_all`	Definition of a recurring report execution schedule and delivery preferences.
`scheduled_report_archive_all`	Historical archive mirror of scheduled_report_header_all

Quality Validation

Score	100/100
Grade	A
Status	■ PASSED
Total Issues	0
Tables Found	35

2. TABLE-BY-TABLE DOCUMENTATION

For each table: purpose, column definitions, when to INSERT, when to UPDATE, and relationships.

1. `unique_id_header_all`

Central registry for all business IDs. One row per entity type.

Column	Type	Notes
`id`	INT	System surrogate key
`table_name`	VARCHAR(100)	Logical table this ID sequence is for
`id_for`	VARCHAR(50)	Entity this ID represents (e.g. USER, VEHICLE, SHIPMENT)
`prefix`	VARCHAR(20)	Business ID prefix (e.g. UI, VHC, SHP)
`last_id`	VARCHAR(15)	Last issued numeric sequence
`business_id`	VARCHAR(20)	Human■readable business ID (e.g. UI■00001)
`domain_code`	VARCHAR(10)	Domain specific code (e.g. LOG■01)
`effective_from`	DATETIME	Date■time from which this ID is valid
`effective_to`	DATETIME	Date■time until which this ID is valid (NULL = indefinite)
`allocation_date`	DATETIME	When the ID was allocated to the entity
`allocated_by`	INT	User ID who performed the allocation
`total_child_records`	INT	Aggregate counter of child records linked to this entity
`last_child_created_on`	DATETIME	Timestamp of the most recent child record creation
`last_child_id`	VARCHAR(20)	Business ID of the most recent child record
`flag_active`	TINYINT(1)	1=active, 0=inactive flag for quick lookup
`status`	INT	1=active, 2=inactive, 3=retired, 4=deleted
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Last modification timestamp

■ **INSERT when**	Once per entity type during initial setup. Example: register 'student_header_all' with prefix 'STU-'.
■ **UPDATE when**	Increment last_id and modified_on every time a new business ID is generated.

2. `factory_reset_all`

Supporting table: factory_reset_all

Column	Type	Notes

`id`	INT	System surrogate key
`app_version`	VARCHAR(20)	Current deployed application version
`maintenance_mode`	TINYINT(1)	1=maintenance mode ON, 0=OFF
`api_throttle_enabled`	TINYINT(1)	1=API throttling active, 0=disabled
`feature_flag_batch_processing`	TINYINT(1)	Enable/disable batch processing subsystem
`feature_flag_real_time_tracking`	TINYINT(1)	Enable/disable real■time shipment tracking
`feature_flag_dynamic_routing`	TINYINT(1)	Enable/disable dynamic route optimisation
`feature_flag_driver_alerts`	TINYINT(1)	Enable/disable driver alert notifications
`feature_flag_cargo_insurance`	TINYINT(1)	Enable/disable optional cargo insurance
`feature_flag_multi_currency`	TINYINT(1)	Enable/disable multi■currency support
`feature_flag_gst_compliance`	TINYINT(1)	Enforce GST breakdown on all billing tables
`max_concurrent_jobs`	INT	Maximum number of concurrent background jobs
`job_retry_limit`	INT	Number of retries before a job is marked failed
`default_retry_interval_seconds`	INT	Default wait time between job retries
`status`	INT	1=active, 2=archived, 3=deleted
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Last modification timestamp

■ **INSERT when**	When the corresponding business event occurs.
■ **UPDATE when**	When the record status or key fields change.

3. `shipment_request_header_all`

Master entity table for Shipment Request records. One row per shipment request entity.

Column	Type	Notes
`id`	INT	System surrogate key
`status`	INT	1=pending, 2=approved, 3=rejected, 4=cancelled, 5=deleted
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Last modification timestamp

`shipment_request_id`	VARCHAR(20)	Human■readable business ID (e.g. SR■00001)
`shipper_id`	INT	FK to shipper_header_all.id
`origin_location`	VARCHAR(255)	Pickup address or location code
`destination_location`	VARCHAR(255)	Delivery address or location code
`weight_kg`	DECIMAL(10,2)	Total weight of the shipment in kilograms
`volume_cbm`	DECIMAL(10,2)	Total volume of the shipment in cubic meters
`declared_value`	DECIMAL(10,2)	Monetary value declared for customs/insurance
`service_level_id`	INT	FK to service_level_header_all.id
`pickup_date`	DATETIME	Requested pickup date and time
`delivery_deadline`	DATETIME	Promised delivery deadline
`special_handling_flag`	TINYINT	1=requires special handling, 0=normal
`insurance_required`	TINYINT	1=insurance purchased, 0=none
`total_items`	INT	Aggregate count of line■item packages in this request
`total_weight_kg`	DECIMAL(12,2)	Aggregate weight of all items (kg)
`last_item_added_on`	DATETIME	Timestamp of the most recent item added to the request
`added_by`	INT	User ID who created this record

■ **INSERT when**	When a new shipment request is registered or onboarded into the system.
■ **UPDATE when**	When shipment request profile information changes. Always copy old row to shipment_request_archive_all first.

---

4. `shipment_request_archive_all`

Historical mirror of shipment_request_header_all. Stores a full copy of every row before it was changed.

Column	Type	Notes
`id`	INT	System surrogate key
`status`	INT	1=pending, 2=approved, 3=rejected, 4=cancelled, 5=deleted
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Last modification timestamp
`shipment_request_id`	VARCHAR(20)	Human■readable business ID (e.g. SR■00001)
`shipper_id`	INT	FK to shipper (customer) entity
`origin_location`	VARCHAR(255)	Pickup address or location code
`destination_location`	VARCHAR(255)	Delivery address or location code

`weight_kg`	DECIMAL(10,2)	Total weight of the shipment in kilograms
`volume_cbm`	DECIMAL(10,2)	Total volume of the shipment in cubic meters
`declared_value`	DECIMAL(10,2)	Monetary value declared for customs/insurance
`service_level_id`	INT	FK to service level (e.g., standard, express)
`pickup_date`	DATETIME	Requested pickup date and time
`delivery_deadline`	DATETIME	Promised delivery deadline
`special_handling_flag`	TINYINT	1=requires special handling, 0=normal
`insurance_required`	TINYINT	1=insurance purchased, 0=none
`total_items`	INT	Aggregate count of line■item packages in this request
`total_weight_kg`	DECIMAL(12,2)	Aggregate weight of all items (kg)
`last_item_added_on`	DATETIME	Timestamp of the most recent item added to the request
`added_by`	INT	User ID who created this record
`archived_on`	DATETIME	When this version was archived
`archived_by`	INT	User ID who performed the archive
`archive_reason`	VARCHAR(255)	Reason for archiving this version

■ INSERT when	Every time a row in shipment_request_header_all is about to be UPDATED. Copy the old row here first.
■ UPDATE when	Never update archive rows — they are immutable historical records.

5. `shipment_header_all`

Master entity table for Shipment records. One row per shipment entity.

Column	Type	Notes
`id`	INT	System surrogate key
`status`	INT	1=created, 2=in_transit, 3=delivered, 4=exception, 5=closed, 6=deleted
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Last modification timestamp
`shipment_id`	VARCHAR(20)	Human■readable business ID (e.g. S■00001)
`shipment_request_id`	INT	Reference to originating shipment request (shipment_request_header_all.id)
`carrier_id`	INT	FK to carrier_header_all.id
`vehicle_id`	INT	FK to vehicle_header_all.id
`driver_id`	INT	FK to driver_header_all.id

`origin_location`	VARCHAR(255)	Actual pickup location
`destination_location`	VARCHAR(255)	Actual delivery location
`weight_kg`	DECIMAL(10,2)	Weight of the shipment (kg)
`volume_cbm`	DECIMAL(10,2)	Volume of the shipment (cbm)
`declared_value`	DECIMAL(10,2)	Declared monetary value
`sgst_amount`	DECIMAL(10,2)	State GST amount
`cgst_amount`	DECIMAL(10,2)	Central GST amount
`igst_amount`	DECIMAL(10,2)	Integrated GST amount
`sgst_percentage`	DECIMAL(5,2)	SGST rate applied
`cgst_percentage`	DECIMAL(5,2)	CGST rate applied
`total_distance_km`	DECIMAL(12,2)	Cumulative distance travelled (km)
`total_fuel_consumed_liters`	DECIMAL(12,2)	Fuel consumed for the shipment
`total_items`	INT	Number of line■item packages in the shipment
`last_status_update_on`	DATETIME	Timestamp of the most recent status change
`expected_delivery_on`	DATETIME	System■calculated expected delivery datetime
`actual_delivery_on`	DATETIME	Actual delivery datetime when completed
`added_by`	INT	User ID who created this record

■ INSERT when	When a new shipment is registered or onboarded into the system.
■ UPDATE when	When shipment profile information changes. Always copy old row to shipment_archive_all first.

6. `shipment_archive_all`

Historical mirror of shipment_header_all. Stores a full copy of every row before it was changed.

Column	Type	Notes
`id`	INT	System surrogate key
`status`	INT	1=created, 2=in_transit, 3=delivered, 4=exception, 5=closed, 6=deleted
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Last modification timestamp
`shipment_id`	VARCHAR(20)	Human■readable business ID (e.g. S■00001)
`shipment_request_id`	VARCHAR(20)	Reference to originating shipment request
`carrier_id`	INT	FK to carrier (logistics provider) entity
`vehicle_id`	INT	FK to vehicle used for the shipment

`driver_id`	INT	FK to driver assigned to the shipment
`origin_location`	VARCHAR(255)	Actual pickup location
`destination_location`	VARCHAR(255)	Actual delivery location
`weight_kg`	DECIMAL(10,2)	Weight of the shipment (kg)
`volume_cbm`	DECIMAL(10,2)	Volume of the shipment (cbm)
`declared_value`	DECIMAL(10,2)	Declared monetary value
`sgst_amount`	DECIMAL(10,2)	State GST amount
`cgst_amount`	DECIMAL(10,2)	Central GST amount
`igst_amount`	DECIMAL(10,2)	Integrated GST amount
`sgst_percentage`	DECIMAL(5,2)	SGST rate applied
`cgst_percentage`	DECIMAL(5,2)	CGST rate applied
`total_distance_km`	DECIMAL(12,2)	Cumulative distance travelled (km)
`total_fuel_consumed_liters`	DECIMAL(12,2)	Fuel consumed for the shipment
`total_items`	INT	Number of line■item packages in the shipment
`last_status_update_on`	DATETIME	Timestamp of the most recent status change
`expected_delivery_on`	DATETIME	System■calculated expected delivery datetime
`actual_delivery_on`	DATETIME	Actual delivery datetime when completed
`added_by`	INT	User ID who created this record
`archived_on`	DATETIME	When this version was archived
`archived_by`	INT	User ID who performed the archive
`archive_reason`	VARCHAR(255)	Reason for archiving this version

■ INSERT when	Every time a row in shipment_header_all is about to be UPDATED. Copy the old row here first.
■ UPDATE when	Never update archive rows — they are immutable historical records.

7. `shipment_life_cycle_all`

Status change audit trail for shipment_header_all. Records every status transition with timestamp.

Column	Type	Notes
`id`	INT	System surrogate key
`shipment_id`	INT	FK to shipment_header_all.id
`previous_status`	INT	Previous status code of the shipment (1=created,2=planned,3=in_transit,4=delivered,5=cancelled)

`new_status`	INT	New status code after transition (same code set as previous_status)
`reason`	VARCHAR(500)	Reason for status change
`changed_by`	INT	User/employee ID who performed the change
`changed_on`	DATETIME	Timestamp when the change occurred
`trigger_event`	VARCHAR(100)	What triggered this transition (e.g. scan, system_job, manual_override)
`remarks`	TEXT	Additional remarks for the transition
`status`	INT	1=active, 2=inactive, 3=deleted
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Last modification timestamp

■ **INSERT when**	Every time the status column in shipment_header_all changes. Record previous_status, new_status, and datetime.
■ **UPDATE when**	Never update life cycle rows — append only.

8. `route_plan_details_all`

Supporting table: route_plan_details_all

Column	Type	Notes
`id`	INT	System surrogate key
`shipment_id`	INT	FK to shipment_header_all.id
`sort_order`	INT	Sequence number of the route plan line item
`carrier_id`	INT	FK to carrier_header_all.id
`vehicle_id`	INT	FK to vehicle_header_all.id
`optimal_path`	VARCHAR(500)	Serialized optimal path (e.g. waypoint IDs or encoded polyline)
`distance_km`	DECIMAL(10,2)	Total distance of this route segment in kilometers
`estimated_time_minutes`	INT	Estimated travel time for this segment in minutes
`alternative_path`	VARCHAR(500)	Serialized alternative path if primary fails
`remarks`	TEXT	Additional notes for the route segment
`status`	INT	1=active, 2=inactive, 3=deleted
`added_by`	INT	User ID who created this route detail
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Last modification timestamp

■ **INSERT when**	When the corresponding business event occurs.

■ **UPDATE when**	When the record status or key fields change.

9. `shipment_event_transaction_all`

Event/ledger table for shipment_event operations. Append-only record of all events.

Column	Type	Notes
`id`	INT	System surrogate key
`shipment_id`	INT	FK to shipment_header_all.id
`month_year`	VARCHAR(7)	YYYY-MM for partitioning and analytics
`transaction_ref`	VARCHAR(30)	Human■readable transaction ID (e.g. SHPEVT-00001)
`event_type`	VARCHAR(50)	Type of event (SCAN, GPS_PING, STATUS_CHANGE, etc.)
`event_timestamp`	DATETIME	Exact time the event was generated
`location_lat`	DECIMAL(9,6)	Latitude of the event location
`location_long`	DECIMAL(9,6)	Longitude of the event location
`status_before`	INT	Shipment status before this event (integer code)
`status_after`	INT	Shipment status after this event (integer code)
`device_id`	VARCHAR(50)	Identifier of the device that generated the event
`ip_address`	VARCHAR(45)	IP address of the source system/device
`amount`	DECIMAL(10,2)	Financial amount associated with the event, if any
`sgst_amount`	DECIMAL(10,2)	State GST amount for the event, if applicable
`cgst_amount`	DECIMAL(10,2)	Central GST amount for the event, if applicable
`igst_amount`	DECIMAL(10,2)	Integrated GST amount for the event, if applicable
`sgst_percentage`	DECIMAL(5,2)	SGST rate applied
`cgst_percentage`	DECIMAL(5,2)	CGST rate applied
`added_by`	INT	User ID who recorded this event
`remarks`	TEXT	Free■form notes about the event
`status`	INT	1=active, 2=inactive, 3=deleted
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Last modification timestamp

■ **INSERT when**	Every time a new transaction, payment, or event occurs. Each row represents one atomic event.

■ **UPDATE when**	Update status column only (e.g. pending → paid). Never modify financial columns after insert.

10. `shipment_tracking_details_all`

Supporting table: shipment_tracking_details_all

Column	Type	Notes
`id`	INT	System surrogate key
`shipment_id`	INT	FK to shipment_header_all.id
`sort_order`	INT	Sequence number for tracking detail rows (e.g., latest first = 1)
`latest_event_type`	VARCHAR(50)	Most recent event type recorded for the shipment
`latest_event_timestamp`	DATETIME	Timestamp of the most recent event
`current_location`	VARCHAR(200)	Human■readable description of current location
`eta_timestamp`	DATETIME	Estimated time of arrival at final destination
`total_distance_km`	DECIMAL(10,2)	Cumulative distance travelled by the shipment in km
`remarks`	TEXT	Additional tracking notes or exceptions
`status`	INT	1=active, 2=inactive, 3=deleted
`added_by`	INT	User ID who generated this tracking summary
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Last modification timestamp

■ **INSERT when**	When the corresponding business event occurs.
■ **UPDATE when**	When the record status or key fields change.

11. `shipment_status_lookup_all`

Supporting table: shipment_status_lookup_all

Column	Type	Notes
`id`	INT	System surrogate key
`status`	INT	1=active, 2=inactive, 3=deleted
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Last modification timestamp
`status_code`	VARCHAR(20)	Short code for status (e.g. IN_TRANSIT)
`status_description`	VARCHAR(255)	Full description of shipment status

`is_terminal`	TINYINT	1=terminal status (Delivered, Cancelled), 0=non■terminal
`requires_customer_notification`	TINYINT	1=notify customer when status reached
`estimated_max_days`	INT	Maximum days expected for this status
`escalation_level`	INT	0=none, 1=low, 2=medium, 3=high
`color_hex`	CHAR(7)	UI colour code for status badge
`display_order`	INT	Order in UI lists
`created_by`	INT	User ID who created this lookup entry
`modified_by`	INT	User ID who last modified this lookup entry

■ **INSERT when**	When the corresponding business event occurs.
■ **UPDATE when**	When the record status or key fields change.

12. `exception_type_lookup_all`

Supporting table: exception_type_lookup_all

Column	Type	Notes
`id`	INT	System surrogate key
`status`	INT	1=active, 2=inactive, 3=deleted
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Last modification timestamp
`exception_code`	VARCHAR(30)	Short code for exception type (e.g. DELAY, DAMAGE)
`exception_name`	VARCHAR(100)	Human readable name of exception type
`description`	TEXT	Detailed description of the exception category
`default_severity`	INT	1=low, 2=medium, 3=high, 4=critical
`is_financial_impact`	TINYINT	1=exception may involve monetary claim
`compensation_policy`	VARCHAR(255)	Reference to compensation policy document
`auto_escalate`	TINYINT	1=auto■escalate to higher authority
`escalation_time_hours`	INT	Hours after which auto■escalation occurs
`created_by`	INT	User ID who created this exception type
`modified_by`	INT	User ID who last modified this exception type

■ **INSERT when**	When the corresponding business event occurs.
■ **UPDATE when**	When the record status or key fields change.

13. `exception_record_transaction_all`

Event/ledger table for exception_record operations. Append-only record of all events.

Column	Type	Notes
`id`	INT	System surrogate key
`status`	INT	1=active, 2=resolved, 3=closed, 4=deleted
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Last modification timestamp
`month_year`	VARCHAR(7)	YYYY-MM for partitioning
`transaction_ref`	VARCHAR(30)	Human■readable transaction ID
`shipment_id`	INT	FK to shipment_header_all.id
`exception_type_id`	INT	FK to exception_type_lookup_all.id
`detected_by_user_id`	INT	User who recorded the exception
`device_id`	VARCHAR(50)	Device identifier used for entry
`ip_address`	VARCHAR(45)	IP address of the device
`exception_timestamp`	DATETIME	When the exception was detected
`severity_level`	INT	1=low, 2=medium, 3=high, 4=critical
`location_latitude`	DECIMAL(9,6)	Latitude of incident location
`location_longitude`	DECIMAL(9,6)	Longitude of incident location
`before_status`	INT	Shipment status before exception (see shipment_status_lookup_all)
`after_status`	INT	Shipment status after exception (see shipment_status_lookup_all)
`affected_weight_kg`	DECIMAL(10,2)	Weight of cargo affected by exception
`affected_value`	DECIMAL(10,2)	Monetary value impacted by exception
`sgst_amount`	DECIMAL(10,2)	State GST amount if financial claim
`cgst_amount`	DECIMAL(10,2)	Central GST amount if financial claim
`igst_amount`	DECIMAL(10,2)	Integrated GST amount if financial claim
`sgst_percentage`	DECIMAL(5,2)	SGST rate applied
`cgst_percentage`	DECIMAL(5,2)	CGST rate applied
`notes`	TEXT	Free■form notes describing the exception
`added_by`	INT	User ID who added this record

■ **INSERT when**	Every time a new transaction, payment, or event occurs. Each row represents one atomic event.
■ **UPDATE when**	Update status column only (e.g. pending → paid). Never modify financial columns after insert.

14. `exception_record_life_cycle_all`

Status change audit trail for exception_record_header_all. Records every status transition with timestamp.

Column	Type	Notes
`id`	INT	System surrogate key
`status`	INT	1=active, 2=inactive, 3=deleted
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Last modification timestamp
`exception_record_id`	INT	FK to exception_record_transaction_all.id
`previous_status`	INT	1=active, 2=resolved, 3=closed, 4=deleted – status before change
`new_status`	INT	1=active, 2=resolved, 3=closed, 4=deleted – status after change
`reason`	VARCHAR(500)	Reason for status change
`changed_by`	INT	User who performed the change
`changed_on`	DATETIME	When the change occurred
`trigger_event`	VARCHAR(100)	What triggered this transition (e.g. USER_ACK, AUTO_ESCALATE)
`remarks`	TEXT	Additional remarks

■ **INSERT when**	Every time the status column in exception_record_header_all changes. Record previous_status, new_status, and datetime.
■ **UPDATE when**	Never update life cycle rows — append only.

---

15. `route_plan_exception_all`

Supporting table: route_plan_exception_all

Column	Type	Notes
`id`	INT	System surrogate key
`status`	INT	1=active, 2=inactive, 3=deleted
`route_plan_exception_id`	VARCHAR(20)	Business ID for route plan exception (e.g. RPE-00001)
`exception_transaction_id`	INT	FK to exception_record_transaction_all.id
`original_route_id`	INT	FK to route_plan_details_all.id (original route)
`alternative_route_id`	INT	FK to route_plan_details_all.id (alternative route)
`generated_by`	INT	User ID who generated the alternative route
`generated_on`	DATETIME	Timestamp when alternative route was generated

`reason`	VARCHAR(500)	Reason for generating alternative route
`priority`	INT	1=high, 2=medium, 3=low priority
`estimated_delay_minutes`	INT	Estimated delay in minutes due to exception
`sgst_amount`	DECIMAL(10,2)	State GST amount associated with any surcharge
`cgst_amount`	DECIMAL(10,2)	Central GST amount associated with any surcharge
`igst_amount`	DECIMAL(10,2)	Integrated GST amount associated with any surcharge
`sgst_percentage`	DECIMAL(5,2)	SGST rate applied
`cgst_percentage`	DECIMAL(5,2)	CGST rate applied
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Last modification timestamp
`added_by`	INT	User ID who created this record

■ INSERT when	When the corresponding business event occurs.
■ UPDATE when	When the record status or key fields change.

---

16. `invoice_transaction_all`

Event/ledger table for invoice operations. Append-only record of all events.

Column	Type	Notes
`id`	INT	System surrogate key
`status`	INT	1=active, 2=inactive, 3=deleted, 4=archived
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Last modification timestamp
`month_year`	VARCHAR(7)	YYYY-MM for partitioning
`transaction_ref`	VARCHAR(30)	Human■readable transaction ID
`invoice_id`	VARCHAR(20)	Business ID of the invoice (e.g. INV■00001)
`shipment_id`	INT	FK to shipment_header_all.id
`shipper_id`	INT	FK to factory_reset_all.id (business entity that ships the goods)
`consignee_id`	INT	FK to factory_reset_all.id (business entity receiving the goods)
`billing_address_id`	INT	Reference to address used for billing
`issue_date`	DATE	Date invoice was issued
`due_date`	DATE	Payment due date
`sub_total_amount`	DECIMAL(10,2)	Sum of line item amounts before taxes
`sgst_amount`	DECIMAL(10,2)	State GST amount

`cgst_amount`	DECIMAL(10,2)	Central GST amount
`igst_amount`	DECIMAL(10,2)	Integrated GST amount
`sgst_percentage`	DECIMAL(5,2)	SGST rate applied
`cgst_percentage`	DECIMAL(5,2)	CGST rate applied
`total_tax_amount`	DECIMAL(10,2)	Total tax (SGST+CGST+IGST)
`total_amount`	DECIMAL(10,2)	Grand total payable
`previous_total_amount`	DECIMAL(10,2)	Previous total before amendment
`after_total_amount`	DECIMAL(10,2)	Total after amendment
`added_by`	INT	User ID who created this record
`device_id`	VARCHAR(50)	Device identifier used at creation
`ip_address`	VARCHAR(45)	IP address of creator

■ **INSERT when**	Every time a new transaction, payment, or event occurs. Each row represents one atomic event.
■ **UPDATE when**	Update status column only (e.g. pending → paid). Never modify financial columns after insert.

17. `invoice_life_cycle_all`

Status change audit trail for invoice_header_all. Records every status transition with timestamp.

Column	Type	Notes
`id`	INT	System surrogate key
`status`	INT	1=active, 2=inactive, 3=deleted
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Last modification timestamp
`invoice_id`	INT	FK to invoice_transaction_all.id
`previous_status`	INT	1=generated, 2=sent, 3=paid, 4=cancelled, 5=adjusted
`new_status`	INT	1=generated, 2=sent, 3=paid, 4=cancelled, 5=adjusted
`reason`	VARCHAR(500)	Reason for status change
`changed_by`	INT	User who made the change
`changed_on`	DATETIME	When the change occurred
`trigger_event`	VARCHAR(100)	What triggered this transition
`remarks`	TEXT	Additional remarks

■ **INSERT when**	Every time the status column in invoice_header_all changes. Record previous_status, new_status, and datetime.
■ **UPDATE when**	Never update life cycle rows — append only.

18. `ledger_entry_transaction_all`

Event/ledger table for ledger_entry operations. Append-only record of all events.

Column	Type	Notes
`id`	INT	System surrogate key
`status`	INT	1=active, 2=inactive, 3=deleted
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Last modification timestamp
`month_year`	VARCHAR(7)	YYYY-MM for partitioning
`transaction_ref`	VARCHAR(30)	Human■readable transaction ID
`ledger_entry_id`	VARCHAR(20)	Business ID of the ledger entry (e.g. LED■00001)
`shipment_id`	INT	FK to shipment_header_all.id
`account_id`	INT	FK to factory_reset_all.id (financial account impacted)
`entry_type`	VARCHAR(20)	Debit or Credit
`entry_amount`	DECIMAL(10,2)	Amount of this ledger entry
`sgst_amount`	DECIMAL(10,2)	State GST amount
`cgst_amount`	DECIMAL(10,2)	Central GST amount
`igst_amount`	DECIMAL(10,2)	Integrated GST amount
`sgst_percentage`	DECIMAL(5,2)	SGST rate applied
`cgst_percentage`	DECIMAL(5,2)	CGST rate applied
`closing_balance`	DECIMAL(12,2)	Running balance after this transaction
`previous_closing_balance`	DECIMAL(12,2)	Balance before this entry
`after_closing_balance`	DECIMAL(12,2)	Balance after this entry
`added_by`	INT	User ID who created this record
`device_id`	VARCHAR(50)	Device identifier used at creation
`ip_address`	VARCHAR(45)	IP address of creator

■ **INSERT when**	Every time a new transaction, payment, or event occurs. Each row represents one atomic event.
■ **UPDATE when**	Update status column only (e.g. pending → paid). Never modify financial columns after insert.

19. `audit_log_transaction_all`

Event/ledger table for audit_log operations. Append-only record of all events.

Column	Type	Notes
`id`	INT	System surrogate key
`audit_uuid`	VARCHAR(36)	UUID primary key for audit log entry
`status`	INT	1=active, 2=inactive, 3=deleted
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Last modification timestamp
`month_year`	VARCHAR(7)	YYYY-MM for partitioning
`transaction_ref`	VARCHAR(30)	Human■readable transaction ID for audit entry
`table_name`	VARCHAR(100)	Name of the table being audited
`row_id`	VARCHAR(36)	Primary key value of the affected row
`action`	VARCHAR(10)	INSERT|UPDATE|DELETE
`before_state`	JSON	JSON snapshot of row before change
`after_state`	JSON	JSON snapshot of row after change
`actor_user_id`	VARCHAR(36)	User who performed the action
`ip_address`	VARCHAR(45)	IP address of the actor
`user_agent`	VARCHAR(255)	User■agent string of the client
`device_id`	VARCHAR(50)	Device identifier used for the action
`added_by`	INT	System user ID that logged this audit entry

■ **INSERT when**	Every time a new transaction, payment, or event occurs. Each row represents one atomic event.
■ **UPDATE when**	Update status column only (e.g. pending → paid). Never modify financial columns after insert.

20. `approval_task_details_all`

Supporting table: approval_task_details_all

Column	Type	Notes
`id`	INT	System surrogate key
`approval_task_id`	VARCHAR(20)	Human■readable business ID for the approval task (e.g. APR-00001)
`exception_id`	INT	FK to exception_record_transaction_all.id
`sort_order`	INT	Sequence number of the task within the approval workflow
`task_name`	VARCHAR(100)	Descriptive name of the approval step
`approver_role`	VARCHAR(50)	Role responsible for this approval (e.g. MANAGER, SUPERVISOR)
`approver_id`	INT	User ID of the assigned approver

`approval_status`	INT	1=pending, 2=approved, 3=rejected, 4=escalated
`approval_comment`	VARCHAR(500)	Approver remarks for this step
`approval_timestamp`	DATETIME	Date■time when the approval decision was recorded
`escalation_level`	INT	Number of times this task has been escalated
`due_date`	DATETIME	Deadline by which the task must be completed
`reminder_sent_flag`	TINYINT	1=reminder sent, 0=not sent
`status`	INT	1=active, 2=inactive, 3=deleted, 4=archived
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Last modification timestamp
`created_by`	INT	User ID who created this record
`modified_by`	INT	User ID who last modified this record

■ **INSERT when**	When the corresponding business event occurs.
■ **UPDATE when**	When the record status or key fields change.

---

21. `user_account_header_all`

Master entity table for User Account records. One row per user account entity.

Column	Type	Notes
`id`	INT	System surrogate key
`status`	INT	1=active, 2=inactive, 3=locked, 4=deleted
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Last modification timestamp
`user_account_id`	VARCHAR(20)	Human■readable business ID (e.g. UA■00001)
`username`	VARCHAR(50)	Login username
`email`	VARCHAR(100)	User e■mail address
`password_hash`	VARCHAR(255)	Hashed password
`first_name`	VARCHAR(50)	First name of the user
`last_name`	VARCHAR(50)	Last name of the user
`phone_number`	VARCHAR(20)	Contact phone number
`role_id`	INT	FK to role definition (role_header_all)
`organization_id`	INT	FK to organization the user belongs to (organization_header_all)

`added_by`	INT	User ID who created this record (FK to user_account_header_all.id)
`last_login_on`	DATETIME	Timestamp of last successful login
`failed_login_attempts`	INT	Consecutive failed login attempts
`is_locked`	INT	1=account locked, 0=not locked
`total_api_credentials`	INT	Aggregate counter of API credentials issued to this user
`last_api_credential_on`	DATETIME	Timestamp of most recent API credential issuance
`address_line1`	VARCHAR(150)	Primary address line
`address_line2`	VARCHAR(150)	Secondary address line
`city`	VARCHAR(100)	City of residence
`state`	VARCHAR(100)	State/Province
`postal_code`	VARCHAR(20)	Postal/ZIP code
`country`	VARCHAR(100)	Country

■ **INSERT when**	When a new user account is registered or onboarded into the system.
■ **UPDATE when**	When user account profile information changes. Always copy old row to user_account_archive_all first.

---

22. `user_account_archive_all`

Historical mirror of user_account_header_all. Stores a full copy of every row before it was changed.

Column	Type	Notes
`id`	INT	System surrogate key (archived version)
`status`	INT	1=active, 2=inactive, 3=locked, 4=deleted
`created_on`	DATETIME	Record creation timestamp (original)
`modified_on`	DATETIME	Last modification timestamp (original)
`user_account_id`	VARCHAR(20)	Human■readable business ID (e.g. UA■00001)
`username`	VARCHAR(50)	Login username
`email`	VARCHAR(100)	User e■mail address
`password_hash`	VARCHAR(255)	Hashed password
`first_name`	VARCHAR(50)	First name of the user
`last_name`	VARCHAR(50)	Last name of the user
`phone_number`	VARCHAR(20)	Contact phone number
`role_id`	INT	FK to role definition (role_header_all)
`organization_id`	INT	FK to organization the user belongs to (organization_header_all)

`added_by`	INT	User ID who created this record (FK to user_account_header_all.id)
`last_login_on`	DATETIME	Timestamp of last successful login
`failed_login_attempts`	INT	Consecutive failed login attempts
`is_locked`	INT	1=account locked, 0=not locked
`total_api_credentials`	INT	Aggregate counter of API credentials issued to this user
`last_api_credential_on`	DATETIME	Timestamp of most recent API credential issuance
`address_line1`	VARCHAR(150)	Primary address line
`address_line2`	VARCHAR(150)	Secondary address line
`city`	VARCHAR(100)	City of residence
`state`	VARCHAR(100)	State/Province
`postal_code`	VARCHAR(20)	Postal/ZIP code
`country`	VARCHAR(100)	Country
`archived_on`	DATETIME	When this version was archived
`archived_by`	INT	User who performed the archive
`archive_reason`	VARCHAR(255)	Reason for archiving this version

■ **INSERT when**	Every time a row in user_account_header_all is about to be UPDATED. Copy the old row here first.
■ **UPDATE when**	Never update archive rows — they are immutable historical records.

---

23. `api_credential_details_all`

Supporting table: api_credential_details_all

Column	Type	Notes
`id`	INT	System surrogate key
`status`	INT	1=active, 2=revoked, 3=expired, 4=deleted
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Last modification timestamp
`user_account_id`	INT	FK to user_account_header_all.id
`sort_order`	INT	Sequence order of credentials for the user
`api_key`	VARCHAR(100)	Public API key
`token`	VARCHAR(255)	Secret token associated with the API key
`expires_on`	DATETIME	Expiration datetime of the credential
`is_active`	INT	1=currently active, 0=inactive
`created_by`	INT	User ID who created the credential (FK to user_account_header_all.id)

`last_used_on`	DATETIME	Timestamp of last successful API call using this credential
`usage_count`	INT	Number of times this credential has been used
`description`	VARCHAR(255)	Human readable description of the credential purpose

■ **INSERT when**	When the corresponding business event occurs.
■ **UPDATE when**	When the record status or key fields change.

24. `notification_event_transaction_all`

Event/ledger table for notification_event operations. Append-only record of all events.

Column	Type	Notes
`id`	INT	System surrogate key
`status`	INT	1=active, 2=inactive, 3=deleted, 4=failed
`month_year`	VARCHAR(7)	YYYY-MM for partitioning
`transaction_ref`	VARCHAR(30)	Human■readable transaction ID for this notification event
`initiator_user_id`	INT	User who triggered the event
`recipient_user_id`	INT	User who should receive the notification
`shipment_id`	INT	Related shipment (FK to shipment_header_all.id)
`event_type`	VARCHAR(50)	Type of event (e.g., STATUS_CHANGE, INVOICE_GENERATED)
`before_status`	INT	Previous status value (integer code)
`after_status`	INT	New status value (integer code)
`invoice_amount`	DECIMAL(10,2)	Financial amount associated with the event, if any
`sgst_amount`	DECIMAL(10,2)	State GST amount (if financial event)
`cgst_amount`	DECIMAL(10,2)	Central GST amount (if financial event)
`igst_amount`	DECIMAL(10,2)	Integrated GST amount (if financial event)
`sgst_percentage`	DECIMAL(5,2)	SGST rate applied
`cgst_percentage`	DECIMAL(5,2)	CGST rate applied
`added_by`	INT	User ID who created this event record
`device_id`	VARCHAR(50)	Device identifier from which the event originated
`ip_address`	VARCHAR(45)	IP address of the source
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Last modification timestamp

■ **INSERT when**	Every time a new transaction, payment, or event occurs. Each row represents one atomic event.
■ **UPDATE when**	Update status column only (e.g. pending → paid). Never modify financial columns after insert.

25. `notification_message_details_all`

Supporting table: notification_message_details_all

Column	Type	Notes
`id`	INT	System surrogate key
`status`	INT	1=queued, 2=sent, 3=failed, 4=cancelled
`event_id`	INT	FK to notification_event_transaction_all.id
`sort_order`	INT	Sequence number of the message for the same event
`template_id`	VARCHAR(30)	Identifier of the message template used
`channel_id`	INT	FK to notification_channel_lookup_all.id
`recipient_user_id`	INT	User who will receive the notification
`recipient_contact`	VARCHAR(255)	Contact detail (email, phone, device token) used for delivery
`payload`	JSON	Rendered message payload (JSON for dynamic fields)
`delivery_status`	INT	0=pending, 1=delivered, 2=failed
`failure_reason`	VARCHAR(500)	Reason for delivery failure, if any
`sent_on`	DATETIME	Timestamp when the message was actually sent
`added_by`	INT	User ID who queued this message
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Last modification timestamp

■ **INSERT when**	When the corresponding business event occurs.
■ **UPDATE when**	When the record status or key fields change.

26. `notification_channel_lookup_all`

Supporting table: notification_channel_lookup_all

Column	Type	Notes
`id`	INT	System surrogate key
`status`	INT	1=active, 2=inactive, 3=deprecated

`channel_code`	VARCHAR(20)	Short code for the channel (e.g., EMAIL, SMS, PUSH)
`channel_name`	VARCHAR(100)	Human readable name of the channel
`description`	VARCHAR(255)	Detailed description of the channel capabilities
`is_active`	TINYINT	1=channel can be used, 0=channel disabled
`max_daily_limit`	INT	Maximum number of messages allowed per day for this channel
`priority`	INT	Priority order when multiple channels are eligible (lower = higher priority)
`provider_name`	VARCHAR(100)	External service provider name (e.g., Twilio, SendGrid)
`provider_endpoint`	VARCHAR(255)	API endpoint URL of the provider
`retry_attempts`	INT	Number of retry attempts on failure
`created_by`	INT	User ID who created this channel record
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Last modification timestamp

■ **INSERT when**	When the corresponding business event occurs.
■ **UPDATE when**	When the record status or key fields change.

27. `report_request_header_all`

Master entity table for Report Request records. One row per report request entity.

Column	Type	Notes
`id`	INT	System surrogate key
`status`	INT	1=pending, 2=processing, 3=completed, 4=failed, 5=cancelled
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Last modification timestamp
`report_request_id`	VARCHAR(20)	Human■readable business ID (e.g. RR-00001)
`report_type`	VARCHAR(50)	Type of report requested (e.g. ShipmentSummary, DriverPerformance)
`output_format`	VARCHAR(20)	Desired output format (PDF, CSV, XLSX, JSON)
`filters`	JSON	JSON object containing filter criteria supplied by the user
`parameters`	TEXT	Additional free■form parameters for the reporting engine
`requested_on`	DATETIME	Timestamp when the request was submitted

`scheduled_run`	DATETIME	If set, the report is scheduled to run at this time
`total_records_expected`	BIGINT	Estimated total records that will be processed for this report
`count_generated`	INT	Number of result rows actually generated
`last_generated_on`	DATETIME	Timestamp of the most recent generation attempt
`priority`	INT	1=low, 2=medium, 3=high – processing priority
`is_async`	TINYINT	1=run asynchronously, 0=run synchronously
`notification_channel_id`	INT	FK to notification_channel_lookup_all.id for delivery
`added_by`	INT	User ID who created this request
`shipment_id`	VARCHAR(20)	Related shipment business ID, if report is shipment■specific
`driver_id`	VARCHAR(20)	Related driver business ID, if report is driver■specific
`vehicle_id`	VARCHAR(20)	Related vehicle business ID, if report is vehicle■specific
`total_items`	INT	Aggregate counter: total line items produced
`count_errors`	INT	Aggregate counter: number of errors encountered during generation
`last_error_on`	DATETIME	Timestamp of the most recent error, if any

■ **INSERT when**	When a new report request is registered or onboarded into the system.
■ **UPDATE when**	When report request profile information changes. Always copy old row to report_request_archive_all first.

28. `report_request_archive_all`

Historical mirror of report_request_header_all. Stores a full copy of every row before it was changed.

Column	Type	Notes
`id`	INT	System surrogate key (archived version)
`status`	INT	Status at time of archive
`created_on`	DATETIME	Record creation timestamp (original)
`modified_on`	DATETIME	Last modification timestamp (original)
`report_request_id`	VARCHAR(20)	Human■readable business ID (e.g. RR-00001)
`report_type`	VARCHAR(50)	Type of report requested
`output_format`	VARCHAR(20)	Desired output format
`filters`	JSON	Filter criteria supplied by the user

`parameters`	TEXT	Additional free■form parameters
`requested_on`	DATETIME	Timestamp when the request was submitted
`scheduled_run`	DATETIME	Scheduled run timestamp, if any
`total_records_expected`	BIGINT	Estimated total records for the report
`count_generated`	INT	Number of result rows generated
`last_generated_on`	DATETIME	Timestamp of most recent generation
`priority`	INT	Processing priority (1=low,2=medium,3=high)
`is_async`	TINYINT	Async flag (1=async,0=sync)
`notification_channel_id`	INT	FK to notification_channel_lookup_all
`added_by`	INT	User ID who created the request
`shipment_id`	VARCHAR(20)	Related shipment business ID
`driver_id`	VARCHAR(20)	Related driver business ID
`vehicle_id`	VARCHAR(20)	Related vehicle business ID
`total_items`	INT	Aggregate counter: total line items produced
`count_errors`	INT	Aggregate counter: number of errors encountered
`last_error_on`	DATETIME	Timestamp of most recent error
`archived_on`	DATETIME	When this version was archived
`archived_by`	INT	User ID who performed the archive
`archive_reason`	VARCHAR(255)	Reason for archiving this version

■ **INSERT when**	Every time a row in report_request_header_all is about to be UPDATED. Copy the old row here first.
■ **UPDATE when**	Never update archive rows — they are immutable historical records.

---

29. `report_result_details_all`

Supporting table: report_result_details_all

Column	Type	Notes
`id`	INT	System surrogate key
`status`	INT	1=active, 2=inactive, 3=deleted
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Last modification timestamp
`report_request_id`	INT	FK to report_request_header_all.id
`sort_order`	INT	Sequence number for ordering result rows
`column_name`	VARCHAR(100)	Name of the data column represented in this row

`column_value`	TEXT	String representation of the column value
`numeric_value`	DECIMAL(10,2)	Numeric value when applicable (e.g., amount, weight)
`date_value`	DATETIME	Date/time value when applicable
`sgst_amount`	DECIMAL(10,2)	State GST amount (if applicable)
`cgst_amount`	DECIMAL(10,2)	Central GST amount (if applicable)
`igst_amount`	DECIMAL(10,2)	Integrated GST amount (if applicable)
`sgst_percentage`	DECIMAL(5,2)	SGST rate applied
`cgst_percentage`	DECIMAL(5,2)	CGST rate applied
`closing_balance`	DECIMAL(12,2)	Running balance after this result row (if ledger■type report)
`added_by`	INT	User ID who triggered the generation of this row

■ **INSERT when**	When the corresponding business event occurs.
■ **UPDATE when**	When the record status or key fields change.

30. `scheduled_report_header_all`

Master entity table for Scheduled Report records. One row per scheduled report entity.

Column	Type	Notes
`id`	INT	System surrogate key
`status`	INT	1=active, 2=inactive, 3=paused, 4=deleted
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Last modification timestamp
`scheduled_report_id`	VARCHAR(20)	Human■readable business ID (e.g. SR-00001)
`report_type`	VARCHAR(50)	Type of report to be generated on schedule
`output_format`	VARCHAR(20)	Desired output format for scheduled deliveries
`schedule_cron`	VARCHAR(100)	Cron expression defining execution frequency
`next_run_on`	DATETIME	Timestamp of the next scheduled execution
`last_run_on`	DATETIME	Timestamp of the most recent execution
`last_success_on`	DATETIME	Timestamp of the most recent successful execution
`last_failure_on`	DATETIME	Timestamp of the most recent failure
`delivery_email`	VARCHAR(255)	Email address to which the report will be sent

`delivery_channel_id`	INT	FK to notification_channel_lookup_all.id for alternative delivery
`is_active`	TINYINT	1=enabled, 0=disabled
`added_by`	INT	User ID who created the scheduled report definition
`shipment_id`	VARCHAR(20)	Related shipment business ID, if schedule is shipment■specific
`driver_id`	VARCHAR(20)	Related driver business ID, if schedule is driver■specific
`vehicle_id`	VARCHAR(20)	Related vehicle business ID, if schedule is vehicle■specific
`total_executions`	INT	Aggregate counter: total number of times the schedule has run
`successful_executions`	INT	Aggregate counter: number of successful executions
`failed_executions`	INT	Aggregate counter: number of failed executions
`last_error_message`	VARCHAR(500)	Error message from the most recent failure, if any
`sgst_amount`	DECIMAL(10,2)	State GST amount (if report includes financial data)
`cgst_amount`	DECIMAL(10,2)	Central GST amount (if report includes financial data)
`igst_amount`	DECIMAL(10,2)	Integrated GST amount (if report includes financial data)
`sgst_percentage`	DECIMAL(5,2)	SGST rate applied
`cgst_percentage`	DECIMAL(5,2)	CGST rate applied
`closing_balance`	DECIMAL(12,2)	Running balance snapshot after each execution (if applicable)

■ INSERT when	When a new scheduled report is registered or onboarded into the system.
■ UPDATE when	When scheduled report profile information changes. Always copy old row to scheduled_report_archive_all first.

31. `scheduled_report_archive_all`

Historical mirror of scheduled_report_header_all. Stores a full copy of every row before it was changed.

Column	Type	Notes
`id`	INT	System surrogate key
`status`	INT	1=active, 2=inactive, 3=deleted, 4=archived
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Last modification timestamp

`scheduled_report_id`	VARCHAR(20)	Human■readable business ID (e.g. SRPT-00001)
`report_name`	VARCHAR(255)	Descriptive name of the report
`report_type`	VARCHAR(100)	Type/category of the report (e.g. shipment_summary, financial)
`schedule_expression`	VARCHAR(100)	Cron■style expression defining schedule
`next_run_on`	DATETIME	Planned next execution datetime
`last_run_on`	DATETIME	Datetime when the report was last executed
`is_active`	TINYINT	1=scheduled, 0=paused
`parameters_json`	TEXT	JSON string of report parameters
`output_format`	VARCHAR(50)	File format of generated report (e.g. CSV, PDF, XLSX)
`destination_path`	VARCHAR(500)	Filesystem or S3 path where report is stored
`generated_by`	INT	User ID who created the schedule
`retention_days`	INT	Number of days to retain generated reports
`priority`	INT	Execution priority (lower = higher priority)
`added_by`	INT	User ID who added this schedule
`archived_on`	DATETIME	Timestamp when this version was archived
`archived_by`	INT	User ID who performed the archive
`archive_reason`	VARCHAR(255)	Reason for archiving this record

■ **INSERT when**	Every time a row in scheduled_report_header_all is about to be UPDATED. Copy the old row here first.
■ **UPDATE when**	Never update archive rows — they are immutable historical records.

---

32. `user_account_life_cycle_all`

Status change audit trail for user_account_header_all. Records every status transition with timestamp.

Column	Type	Notes
`id`	INT	System surrogate key
`status`	INT	1=active, 2=inactive, 3=deleted
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Last modification timestamp
`user_account_id`	INT	FK to user_account_header_all.id
`previous_status`	INT	Previous account status
`new_status`	INT	New account status
`reason`	VARCHAR(500)	Reason for status change
`changed_by`	INT	User who performed the change

`changed_on`	DATETIME	When the change occurred
`trigger_event`	VARCHAR(100)	What triggered this transition
`remarks`	TEXT	Additional remarks

■ **INSERT when**	Every time the status column in user_account_header_all changes. Record previous_status, new_status, and datetime.
■ **UPDATE when**	Never update life cycle rows — append only.

---

33. `shipment_request_life_cycle_all`

Status change audit trail for shipment_request_header_all. Records every status transition with timestamp.

Column	Type	Notes
`id`	INT	System surrogate key
`status`	INT	1=active, 2=inactive, 3=deleted
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Last modification timestamp
`shipment_request_id`	INT	FK to shipment_request_header_all.id
`previous_status`	INT	Previous lifecycle status
`new_status`	INT	New lifecycle status
`reason`	VARCHAR(500)	Reason for status change
`changed_by`	INT	User who performed the change
`changed_on`	DATETIME	When the change occurred
`trigger_event`	VARCHAR(100)	What triggered this transition
`remarks`	TEXT	Additional remarks

■ **INSERT when**	Every time the status column in shipment_request_header_all changes. Record previous_status, new_status, and datetime.
■ **UPDATE when**	Never update life cycle rows — append only.

---

34. `scheduled_report_life_cycle_all`

Status change audit trail for scheduled_report_header_all. Records every status transition with timestamp.

Column	Type	Notes
`id`	INT	System surrogate key
`status`	INT	1=active, 2=inactive, 3=deleted
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Last modification timestamp
`scheduled_report_id`	INT	FK to scheduled_report_header_all.id

`previous_status`	INT	Previous lifecycle status
`new_status`	INT	New lifecycle status
`reason`	VARCHAR(500)	Reason for status change
`changed_by`	INT	User who performed the change
`changed_on`	DATETIME	When the change occurred
`trigger_event`	VARCHAR(100)	What triggered this transition
`remarks`	TEXT	Additional remarks

■ **INSERT when**	Every time the status column in scheduled_report_header_all changes. Record previous_status, new_status, and datetime.
■ **UPDATE when**	Never update life cycle rows — append only.

---

35. `report_request_life_cycle_all`

Status change audit trail for report_request_header_all. Records every status transition with timestamp.

Column	Type	Notes
`id`	INT	System surrogate key
`status`	INT	1=active, 2=inactive, 3=deleted
`created_on`	DATETIME	Record creation timestamp
`modified_on`	DATETIME	Last modification timestamp
`report_request_id`	INT	FK to report_request_header_all.id
`previous_status`	INT	Previous lifecycle status
`new_status`	INT	New lifecycle status
`reason`	VARCHAR(500)	Reason for status change
`changed_by`	INT	User who performed the change
`changed_on`	DATETIME	When the change occurred
`trigger_event`	VARCHAR(100)	What triggered this transition
`remarks`	TEXT	Additional remarks

■ **INSERT when**	Every time the status column in report_request_header_all changes. Record previous_status, new_status, and datetime.
■ **UPDATE when**	Never update life cycle rows — append only.

---

3. PRODUCTION RULES APPLIED

The following 28 rules from the AI Database Rule Framework were applied during schema generation.

Rule ID	Rule Name	Category	Priority
1	Naming Taxonomy	naming	CRITICAL
7	Indian GST Compliance	taxation	CRITICAL
9	Centralised ID Registry	identity	CRITICAL
2	Two-ID System	identity	CRITICAL
98	Three-Table State Machine for Approvals	workflow	HIGH
45	Temp-to-Permanent Promotion	architecture	HIGH
11	Flat Approval Chains	workflow	HIGH
51	Failed Transactions in Separate Table	financial	HIGH
54	Monthly Calendar Table Pattern	calendar	HIGH
103	Multi-Tier Transaction Consolidation	financial	HIGH
3	Three-Layer Data Preservation	audit	HIGH
30	Timestamp Column Naming	naming	HIGH
38	Aggregate Counters on Parent Rows	performance	HIGH
39	Snapshot + Event Log Dual Table	architecture	HIGH
52	DB-Backed Async Job Queue	operations	HIGH
102	Error Logging as Schema Entity with Lifecycle	operations	MEDIUM
10	Infrastructure Monitoring via Schema	monitoring	MEDIUM
86	Targets and Actuals in Separate Tables	financial	MEDIUM
92	Flat Tables for Fixed Third-Party API Responses	domain	MEDIUM
36	Prefixed Column Naming for JOIN-Heavy Schemas	naming	MEDIUM
8	Status Integer Convention	convention	MEDIUM
94	Operational State on Reusable Entity Row	architecture	MEDIUM
27	Denormalised Running Balance	financial	HIGH
29	Decimal Money Fields	financial	CRITICAL
57	Pre-Computed Analytics at Write Time	financial	HIGH
18	Comprehensive Audit Logging	audit	HIGH
31	Named FK Constraints	constraints	HIGH
32	Named Indexes	constraints	HIGH

4. DEVELOPER NOTES

Business ID Generation	All business IDs (student_id, fee_id, etc.) must be generated from unique_id_header_all. Insert a row to register each entity type with its prefix. Increment last_id on every new record. Never use the auto-increment 'id' column as a business identifier.
Foreign Key References	All foreign keys must reference the integer 'id' PRIMARY KEY column, not the business ID (varchar) column. The business ID is for display only. Join tables using integer id columns for performance.
Archive Table Usage	Copy the full row to the corresponding _archive_all table BEFORE making any UPDATE to a _header_all table. Archive tables store the history of every change. Never DELETE from _header_all — use status instead.
Life Cycle Table Usage	Insert a row into _life_cycle_all every time the status column changes. Store previous_status, new_status, and the datetime of the change. This enables full audit trail of every state transition.
GST Handling	Always store CGST and SGST amounts separately. Never combine them into a single tax column. Calculate at write time and store — never compute at read time. Current rate: CGST 9% + SGST 9% = 18% total (verify with your accountant for current applicable rates).
Closing Balance	The closing_balance column on transaction tables must be updated atomically with the transaction insert. Use a database transaction (BEGIN/COMMIT) to ensure the balance and the transaction record are always consistent. Never update balance separately.
Status Convention	Status 1 = active/success. Status 2 = inactive/failure. Status 3 = deleted/cancelled. Never hard DELETE rows. Set status = 2 or 3 and use soft delete everywhere.
Index Usage	All foreign key columns are indexed. Use the named indexes in your WHERE clauses. Avoid SELECT * in production — always select specific columns to use covering indexes effectively.